DEVOPS HANDS-ON LAB GUIDE · BEGINNER FRIENDLY

Build Lean & Secure Docker Images

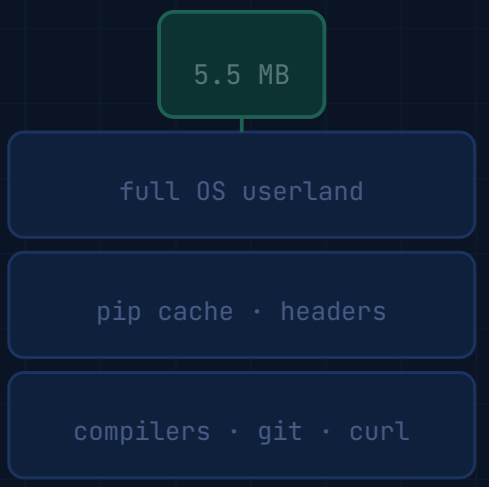
A from-scratch, step-by-step guide to everything covered in the live session: slim base images, multi-stage builds, a 5 MB Go service on `scratch`, image hardening, runtime lockdown and vulnerability scanning with Trivy. Every step shows the command, the output you should see, and what to do when it doesn't work.

Docker · Multi-stage builds · Python / Flask · Go · Non-root containers · Linux capabilities · Trivy

~1 GB → ~140 MB
Python API: full image vs slim multi-stage

~850 MB → 5.5 MB
Go service: builder image vs final scratch image

root → app
Hardened, read-only, capability-free container



Session	Trainer	Level	Lab time
Build Lean & Secure Images using Docker	**Rajan**	**Beginner → Intermediate**	**~90 minutes**

CONTENTS

What's inside

01	Why lean and secure images matter The business problems this session solves	3
02	Docker basics from absolute scratch Image, container, layer, base image, tag, registry, build context	5
03	Lab setup Install check, pre-pulls, the size activity, project folders	7
04	Lab 1 · Python Product API: single-stage vs multi-stage Build both, compare sizes, run and test	10
05	Lab 2 · Go Inventory service: ~850 MB to 5.5 MB Static binaries and the empty scratch image	16
06	Session Q&A · Making an image from a running container docker commit vs Dockerfile, pushing to Docker Hub	20
07	Image hardening · The Ten Rules The checklist every production image should pass	22
08	Lab 3 · Build the hardened Product API image Non-root user, healthcheck, clean Python env	24
09	Lab 4 · Lock the container down at runtime Read-only FS, dropped capabilities, resource limits	28
10	Lab 5 · Scan for vulnerabilities with Trivy Run the scan, read the report, fix what it finds	31
11	Troubleshooting index Every error from this lab, its cause and its fix	34
12	Cheat sheet, cleanup and practice tasks Commands at a glance and exercises to try on your own	36

How to read this guide

Do the labs in order: each one builds on files created in the previous one. Type the commands (or copy them carefully) into the VS Code terminal and compare what you see with the output blocks.

OUTPUT What your terminal should show. IDs, timestamps and exact sizes will differ slightly on your machine.

CONCEPT The idea behind the step, explained in plain words.

IF IT FAILS The most likely errors for that step and how to fix them.

WINDOWS Notes for PowerShell / Docker Desktop on Windows (what most of us used in the session).

CHECKPOINT Confirm this before moving on.

DANGER Things you should never do in a real project.

PART 01

Why lean and secure images matter

A Docker image is what actually runs in production. If it's huge, deployments are slow. If it's insecure, one small bug in your app can hand an attacker the whole container. This session fixes both.

The production view

PROBLEM	BUSINESS IMPACT	FIXED IN THIS GUIDE BY
1 GB images	Slow deployments, slow autoscaling, higher registry and network cost	Slim base + multi-stage builds (Labs 1–2)
Running as root	A single app bug can become full control of the container	Non-root `USER` (Lab 3)
Secrets inside images	Anyone who can pull the image can steal the key	.dockerignore + runtime secrets (Part 7)
Slow builds	Developers wait, CI minutes cost money	Layer ordering + small bases (Labs 1–2)
Unknown vulnerabilities	Audit failures and breaches	Trivy scanning (Lab 5)

CONCEPT · WHY SIZE IS A REAL COST

When Kubernetes (or ECS, or any orchestrator) scales your app from 2 to 20 copies, every new server must download the image first. A 1 GB image over a busy network can take a minute or more before the app even starts; a 140 MB image takes seconds. You also pay for storing every version in the registry (ECR, Docker Hub) and for the network traffic every time it's pulled.

What you'll build

Two tiny services for an imaginary shop called **ShopEase**:

PRODUCT-API · PYTHON + FLASK

Returns a list of products at `/products` and a health status at `/health`. Runs on port 8000 with Gunicorn.

Used for: slim vs full base, single vs multi-stage, hardening, runtime lockdown, Trivy scan.

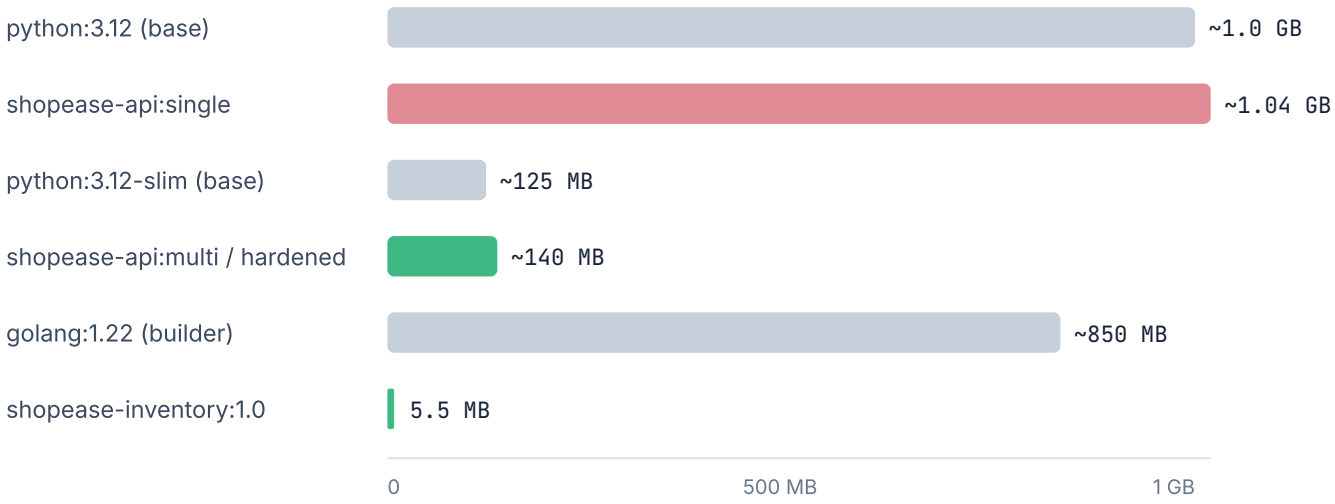
INVENTORY · GO

Returns stock counts at `/stock` and a health status at `/health`. Runs on port 8080.

Used for: compiling to one static binary and shipping it on an empty `scratch` image.

Your final scoreboard

These are the approximate image sizes you'll see by the end. The exact numbers depend on the current base image versions, but the difference will be just as dramatic.



Grey = base/builder images you pull. Red = the naive build. Green = what you actually ship after this session. The Go bar is not a mistake: it really is that small.

PART 02

Docker basics from absolute scratch

If you already know images, containers and layers, skim this. If not, these eight ideas are all you need to understand every command in the labs.

1. Image vs container

An **image** is a read-only package that contains everything your app needs to run: an operating-system userland, a language runtime (Python, Go...), libraries and your code. A **container** is a running copy of an image. One image can start many containers.

> **ANALOGY**
> An image is a recipe printed in a cookbook; a container is the dish you cooked from it. You can cook the same recipe ten times (ten containers), and spilling something on one plate doesn't change the recipe.

2. Dockerfile

A plain text file with step-by-step instructions to build an image. The most common instructions:

INSTRUCTION	WHAT IT DOES
`FROM`	Starting point (the base image). Every Dockerfile starts here.
`WORKDIR`	Creates a folder inside the image and makes it the current directory for the next steps.
`COPY`	Copies files from your computer (the build context) into the image.
`RUN`	Runs a command while building, e.g. installing packages.
`ENV`	Sets an environment variable inside the image.
`USER`	Which Linux user the container runs as from this point on.
`EXPOSE`	Documents which port the app listens on. It does **not** open the port; `-p` on `docker run` does that.
`HEALTHCHECK`	A command Docker runs periodically to check the app is really working.
`CMD` / `ENTRYPOINT`	The command that starts when a container is created from the image.

3. Layers

Each `RUN`, `COPY` and `ADD` instruction creates a new **layer**, a snapshot of the files it added or changed. The image is all the layers stacked together, so the image size is roughly the sum of its layers.

> **IMPORTANT CONSEQUENCE**
> Deleting a file in a later layer does **not** make the image smaller. The file still lives in the earlier layer, just hidden. That's why "install compilers, build, then `rm` them" doesn't work, and why we use **multi-stage builds** instead (Lab 1).

4. Base image and its flavours

The image you start `FROM`. It decides most of your final size and most of your vulnerabilities.

FLAVOUR	EXAMPLE	WHAT'S INSIDE	APPROX. SIZE
Full	`python:3.12`	Full Debian + compilers (gcc), git, curl, headers. Great for building, too heavy to ship.	~1 GB
Slim	`python:3.12-slim`	Minimal Debian + Python. No compilers or extra tools.	~125 MB
Alpine	`python:3.12-alpine`	Tiny Alpine Linux (musl libc). Very small, but some Python packages need extra work.	~50 MB
Distroless	`gcr.io/distroless/…`	Only the runtime. No shell, no package manager.	~20–50 MB
scratch	`scratch`	Completely empty. Only works for self-contained (static) binaries like Go.	0 MB

5. Tags and registries

An image name looks like `repository:tag`, e.g. `python:3.12-slim`. The **tag** is a label for a version. A **registry** (Docker Hub, Amazon ECR…) stores images: `docker pull` downloads, `docker push` uploads.

BEGINNER TRAP · THE LATEST TAG

`latest` is just a default name, not a promise of "newest" or "stable". If you build `FROM python:latest` today and again next month, you can get a completely different Python version and your build may break. Always pin a version like `python:3.12-slim`.

6. Build context (the dot at the end)

In `docker build -t myimage .` the final `.` means "send the current folder to Docker". Only files inside this folder can be `COPY`-ed. A `.dockerignore` file lists what to leave out (like `.git` or `.env`).

7. Port mapping

A container has its own private network. `-p 8080:8000` means "traffic arriving at port **8080 on my laptop** goes to port **8000 inside the container**". Format is always `HOST:CONTAINER`.

8. Root inside a container

By default, processes in a container run as `root` (user ID 0). Containers are isolated, but root inside a container still has far more power than it needs: it can install tools, modify files and try kernel-level tricks to escape. Hardening (Labs 3–4) removes that power.

PART 03

Lab setup

Get your tools ready, download the big images once on good Wi-Fi, and create the project folders.

Prerequisites

TOOL	WHY YOU NEED IT	CHECK
Docker Desktop (Windows / macOS) or Docker Engine (Linux)	Builds and runs the images	Whale icon says "Engine running"
VS Code	Create files and use the built-in terminal	Opens normally
A web browser	Test the APIs	-
~4 GB free disk	Base images + your builds	`docker system df`

You do **not** need Python or Go installed on your laptop. Everything is built inside Docker.

STEP 1 Confirm Docker works

Open VS Code, then open a terminal with **Terminal → New Terminal** (shortcut `Ctrl + ``). Run:

terminal — POWERSHELL / BASH

```
docker version
docker run --rm hello-world
```

EXPECTED OUTPUT — version numbers will differ

```
Client:
 Version:           27.x / 28.x / 29.x
 ...
Server: Docker Desktop
 Engine:
  Version:          27.x / 28.x / 29.x
 ...
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

IF IT FAILS

- `error during connect ... dockerDesktopLinuxEngine: The system cannot find the file specified` or `Cannot connect to the Docker daemon`: Docker Desktop isn't running. Start it and wait until the status shows *Engine running*.
- `docker : The term 'docker' is not recognized`: Docker isn't installed, or VS Code was opened before installing it. Close and reopen VS Code.
- You see only the `Client` section and an error under `Server`: same as the first point, the engine is not running.

STEP 2 Pre-pull the images (do this on good Wi-Fi)

These are big downloads. Pulling them once now means every build later is fast.

terminal — POWERSHELL / BASH

```
docker pull python:3.12
docker pull python:3.12-slim
docker pull golang:1.22
docker pull aquasec/trivy:latest
```

IMAGE	WHAT IT IS
`python:3.12`	Full Python image (Debian + build tools). Used for the "naive" single-stage build.
`python:3.12-slim`	Minimal Python image. Used for the lean and hardened builds.
`golang:1.22`	The Go programming language toolchain image, used only to *compile* the Go service.
`aquasec/trivy:latest`	Contains the Trivy tool, used for scanning Docker images for vulnerabilities.

EXPECTED OUTPUT (PER IMAGE)

```
3.12: Pulling from library/python
a1b2c3d4e5f6: Pull complete
...
Digest: sha256:…
Status: Downloaded newer image for python:3.12
docker.io/library/python:3.12
```

IF IT FAILS

- `toomanyrequests: You have reached your pull rate limit` : Docker Hub limits anonymous pulls. Run `docker login` with a free Docker Hub account and retry.
- `TLS handshake timeout` / `i/o timeout` : network problem. Retry; switch networks; if on office VPN, try without it.
- `no space left on device` : free space with `docker system prune` (removes stopped containers and unused data).

STEP 3 Activity · observe the size of python:3.12 vs python:3.12-slim

terminal — POWERSHELL / BASH

```
docker images python
```

EXPECTED OUTPUT — sizes approximate

```
REPOSITORY   TAG         IMAGE ID       CREATED       SIZE
python       3.12-slim   5c0e1f2a3b4d   2 weeks ago   ~125MB
python       3.12        9a8b7c6d5e4f   2 weeks ago   ~1.02GB
```

The slim image is about **8 times smaller**. Where did the extra ~900 MB go? See for yourself:

terminal — POWERSHELL / BASH

```
docker run --rm python:3.12      sh -c "which gcc git curl"
docker run --rm python:3.12-slim sh -c "which gcc git curl"
```

EXPECTED OUTPUT

```
# full image: all three tools exist
/usr/bin/gcc
/usr/bin/git
/usr/bin/curl
# slim image: prints nothing, the tools are not there
```

CONCEPT

Compilers, git and curl are handy while *building*, but in production they're dead weight and a gift to attackers (they can download and compile tools). Your app only needs Python and its libraries at runtime.

WINDOWS · NEWER DOCKER DESKTOP SHOWS A DIFFERENT TABLE

Recent Docker Desktop versions (with the containerd image store) may print columns like `DISK USAGE` and `CONTENT SIZE` instead of `SIZE`. *Content size* is the compressed download size; *disk usage* is the unpacked size. Just compare the same column across images: the ratio is what matters.

STEP 4 Create the project folders

In VS Code choose **File → Open Folder**, create a new folder called `lean-secure` and open it. Then in the terminal:

Windows — POWERSHELL

```
mkdir product-api, inventory
cd product-api
```

macOS / Linux — BASH

```
mkdir product-api inventory
cd product-api
```

By the end of the session your folder will look like this:

folder structure — TREE

```
lean-secure/
├── product-api/
│   ├── app.py
│   ├── requirements.txt
│   ├── .dockerignore          (recommended)
│   ├── Dockerfile.single
│   ├── Dockerfile.multi
│   └── Dockerfile.hardened
└── inventory/
    ├── go.mod
    ├── main.go
    └── Dockerfile
```

WINDOWS · AVOID ONEDRIVE FOLDERS

If your `Documents` folder is synced by OneDrive, files can be locked or placeholder-only while syncing, which causes odd build errors. Prefer a plain path such as `C:\labs\lean-secure` or `D:\lean-secure`.

PART 04 · LAB 1

Python Product API: single-stage vs multi-stage

Objective: package a small Flask API twice, once the naive way on `python:3.12` and once the lean way on `python:3.12-slim` with a multi-stage build, then compare.

WHERE YOU SHOULD BE

Terminal inside `lean-secure/product-api`. Check with `pwd` (PowerShell or bash). The prompt should end in `…\lean-secure\product-api>`.

STEP 1 Create app.py

In VS Code's Explorer, right-click `product-api` → **New File** → name it exactly `app.py` (all lowercase). Paste:

product-api/app.py — PYTHON

```python
from flask import Flask, jsonify

app = Flask(__name__)

PRODUCTS = [
    {"id": 1, "name": "Laptop Stand", "price": 1499},
    {"id": 2, "name": "USB-C Hub", "price": 2299},
]


@app.get("/health")
def health():
    return {"status": "ok"}


@app.get("/products")
def products():
    return jsonify(PRODUCTS)
```

TWO THINGS THAT BREAK THIS STEP

- **The file name must be `app.py`, not `App.py`.** Windows doesn't care about capital letters, but Linux inside the container does. The start command `app:app` looks for a file called exactly `app.py`.
- **Indentation matters in Python.** The `return` lines must be indented with 4 spaces. If copying from this PDF loses the spaces, re-indent them by hand.

CODE	WHAT IT MEANS
`app = Flask(__name__)`	Creates the web application object, named `app`.
`PRODUCTS = [...]`	Our "database": a hard-coded list of two products.
`@app.get("/health")`	When someone opens `/health`, run the function below. Used by health checks.
`@app.get("/products")`	Returns the product list as JSON.

STEP 2 Create requirements.txt

product-api/requirements.txt — TEXT

```
flask==3.0.3
gunicorn==22.0.0
```

Flask is the web framework. **Gunicorn** is a production web server that runs the Flask app (Flask's built-in server is only for development). Versions are pinned with `==` so every build installs exactly the same thing.

STEP 3 (Recommended) Create .dockerignore

product-api/.dockerignore — TEXT

```
.git
.venv
venv
__pycache__
*.pyc
.env
```

This keeps junk and secrets (like `.env`) out of the build context. Not required for the lab to work, but a habit worth building now.

STEP 4 Create Dockerfile.single (the naive, single-stage way)

product-api/Dockerfile.single — DOCKERFILE

```
FROM python:3.12
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

LINE	WHAT IT DOES
`FROM python:3.12`	Starts from the full ~1 GB Python image.
`WORKDIR /app`	Creates `/app` inside the image and moves into it.
`COPY . .`	Copies *everything* in the folder (app, requirements, Dockerfiles, anything else) into `/app`.
`RUN pip install -r requirements.txt`	Installs Flask and Gunicorn. pip also keeps a download cache, which stays in the image.
`CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]`	Starts Gunicorn on port 8000. `app:app` = "file `app.py`, variable `app`".

CONCEPT · WHY 0.0.0.0 AND NOT LOCALHOST?

Inside a container, `localhost` means "only this container". If Gunicorn listened on `127.0.0.1`, the port mapping from your laptop could never reach it. `0.0.0.0` means "listen on all network interfaces", which is what makes `-p` work.

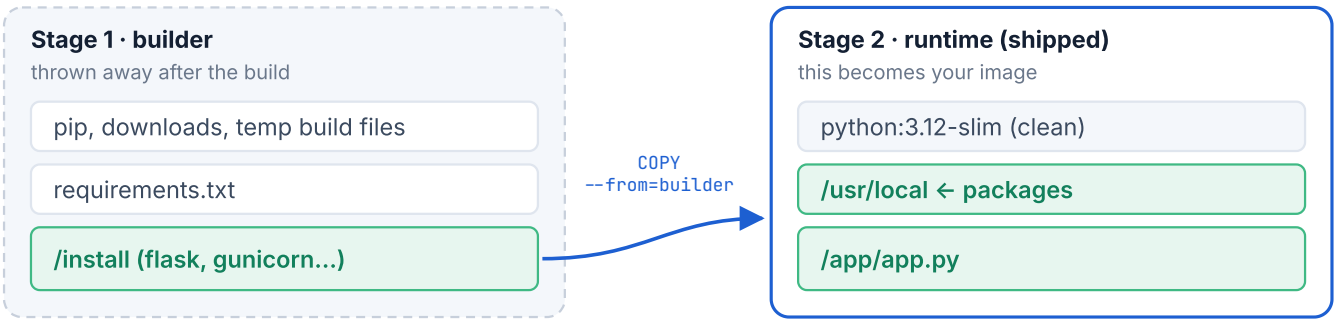
STEP 5 Create Dockerfile.multi (the lean, multi-stage way)

product-api/Dockerfile.multi — DOCKERFILE

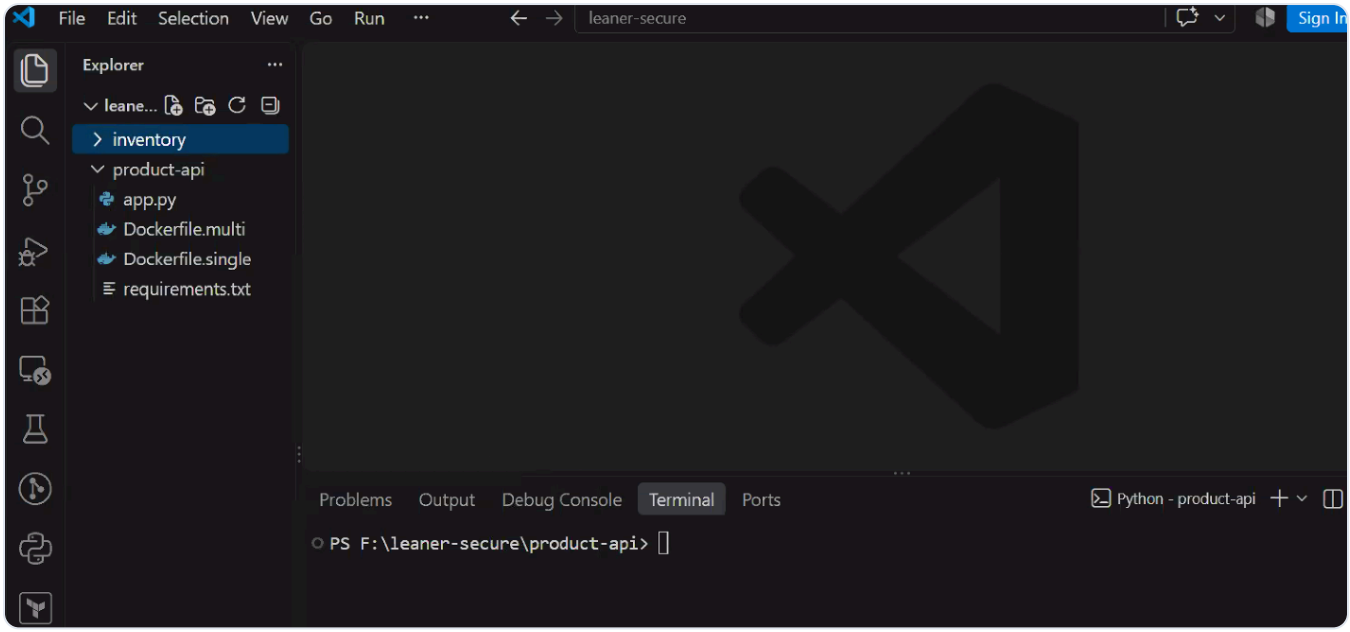
```dockerfile
# ---------- Stage 1: builder ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- Stage 2: runtime ----------
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
EXPOSE 8000
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

LINE	WHAT IT DOES
`FROM python:3.12-slim AS builder`	Stage 1, nicknamed `builder`. A temporary workshop.
`COPY requirements.txt .`	Copies only the requirements, not the code (see "layer caching" below).
`pip install --no-cache-dir --prefix=/install ...`	`--no-cache-dir`: don't keep pip's download cache. `--prefix=/install`: put all installed packages in one folder, `/install`, so they're easy to pick up.
`FROM python:3.12-slim`	Stage 2 starts from a **fresh, clean** slim image. Nothing from stage 1 comes along unless we copy it.
`COPY --from=builder /install /usr/local`	Copies just the installed packages from the builder into `/usr/local`, where Python looks for libraries and where the `gunicorn` command lands.
`COPY app.py .`	Copies only the one file the app needs.
`EXPOSE 8000`	Documents the port (does not publish it).



Multi-stage build: do the messy work in a throwaway stage, then copy only the finished result into a clean final image.



From the session: this is how VS Code looks once Steps 1–5 are done. The product-api folder holds app.py, Dockerfile.multi, Dockerfile.single and requirements.txt, and the terminal is pointing at product-api.

STEP 6 Build both images

terminal · inside product-api — POWERSHELL / BASH

```
docker build -f Dockerfile.single -t shopease-api:single .
docker build -f Dockerfile.multi  -t shopease-api:multi  .
```

`-f` chooses which Dockerfile to use (needed because ours aren't named plain `Dockerfile`). `-t` gives the image a name and tag. The final `.` is the build context: don't forget it.

EXPECTED OUTPUT (MULTI BUILD, TRIMMED)

```
[+] Building 14.2s (12/12) FINISHED
 => [internal] load build definition from Dockerfile.multi
 => [internal] load metadata for docker.io/library/python:3.12-slim
 => [builder 1/4] FROM docker.io/library/python:3.12-slim
 => [builder 2/4] WORKDIR /build
 => [builder 3/4] COPY requirements.txt .
 => [builder 4/4] RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
 => [stage-1 2/4] WORKDIR /app
 => [stage-1 3/4] COPY --from=builder /install /usr/local
 => [stage-1 4/4] COPY app.py .
 => exporting to image
 => => naming to docker.io/library/shopease-api:multi
```

Notice the two names in the build log: `builder` steps and `stage-1` steps. That's the two stages running.

STEP 7 Compare the sizes

terminal — POWERSHELL / BASH

```
docker images shopease-api
```

EXPECTED OUTPUT — sizes approximate

```
REPOSITORY     TAG      IMAGE ID       CREATED          SIZE
shopease-api   multi    7e1d2c3b4a59   10 seconds ago   ~140MB
shopease-api   single   2b3c4d5e6f70   40 seconds ago   ~1.04GB
```

CHECKPOINT

The multi image should be roughly **85–87% smaller** than the single image. Same app, same features.

CONCEPT · THE TWO APPROACHES USED TO BUILD LEAN IMAGES

1. **Slim base image**: switching `python:3.12` → `python:3.12-slim` gives most of the saving here (~900 MB).
2. **Multi-stage build**: only the finished packages reach the final image. For this small Python app the extra saving is modest, but when a build needs compilers (Go, Java, Node, or Python packages with C code) multi-stage is where the big win is. Lab 2 proves it.

STEP 8 Run and test the lean image

terminal — POWERSHELL / BASH

```
docker run -d --name api-multi -p 8001:8000 shopease-api:multi
docker ps
curl.exe http://localhost:8001/products
```

`-d` runs it in the background, `--name` gives the container a friendly name, `-p 8001:8000` maps laptop port 8001 to container port 8000. You can also just open `http://localhost:8001/products` in your browser.

EXPECTED OUTPUT

```
CONTAINER ID   IMAGE                STATUS         PORTS                    NAMES
c0ffee123456   shopease-api:multi   Up 5 seconds   0.0.0.0:8001->8000/tcp   api-multi
(COMMAND and CREATED columns trimmed to fit the page)

[{"id":1,"name":"Laptop Stand","price":1499},{"id":2,"name":"USB-C Hub","price":2299}]
```

WINDOWS · USE CURL.EXE, NOT CURL

In Windows PowerShell 5.1, `curl` is an alias for `Invoke-WebRequest` and prints a big object instead of the JSON. Type `curl.exe` to use the real curl, or simply test in the browser.

Clean up this test container (we'll start fresh ones in Lab 3):

terminal — POWERSHELL / BASH

```
docker rm -f api-multi
```

IF LAB 1 FAILS

`failed to read dockerfile: open Dockerfile.multi: no such file or directory`	You're in the wrong folder (`cd product-api`), or Windows saved the file as `Dockerfile.multi.txt` . Check the exact name in VS Code's Explorer.
`"docker build" requires exactly 1 argument`	You forgot the `.` at the end of the build command.
`COPY failed: ... "/app.py": not found`	The file is named `App.py` or is in another folder. Rename to `app.py` .
`ERROR: Could not find a version that satisfies the requirement ...`	Typo in `requirements.txt` (e.g. `flask=3.0.3` with one `=`) or no internet during build.
Container not in `docker ps` after `docker run`	It crashed. Run `docker ps -a` and `docker logs api-multi` to see why.
Logs show `Failed to find attribute 'app' in 'app'` or `No module named 'app'`	File isn't called `app.py` , or the Flask variable isn't called `app` .
Browser: `ERR_CONNECTION_REFUSED`	Wrong port in the URL, missing `-p` , or the container isn't running.

PART 05 · LAB 2

Go Inventory service: ~850 MB to 5.5 MB

Go can compile your whole program into one self-contained file. Combine that with a multi-stage build and the empty `scratch` image, and the final image is basically just your program.

STEP 1 Move the terminal into the inventory folder

terminal — POWERSHELL / BASH

```
cd ..\inventory      # Windows PowerShell
cd ../inventory      # macOS / Linux
```

EXPECTED PROMPT

```
PS C:\labs\lean-secure\inventory>
```

COMMON MISTAKE

Building while still in `product-api`. Docker then can't find `main.go`, or worse, builds the wrong Dockerfile. Always check the prompt (or run `pwd`) before `docker build`.

STEP 2 Create go.mod

inventory/go.mod — GO MODULE

```
module inventory

go 1.22
```

This tells Go "this folder is a module called `inventory`, written for Go 1.22". It must match (or be lower than) the Go version in the builder image.

STEP 3 Create main.go

inventory/main.go GO

```go
package main

import (
    "fmt"
    "log"
    "net/http"
)

func main() {
    http.HandleFunc("/health", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, `{"status":"ok"}`)
    })
    http.HandleFunc("/stock", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprint(w, `{"laptop-stand":42,"usb-c-hub":17}`)
    })
    log.Println("inventory listening on :8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}
```

Two endpoints, `/health` and `/stock`, served on port 8080 using only Go's standard library (no external packages, so no `go.sum` file is needed). Keep the opening `{` on the same line as `func(...)`: Go refuses to compile if it's moved to the next line.

STEP 4 Create the Dockerfile

inventory/Dockerfile DOCKERFILE

```dockerfile
# ---------- Stage 1: compile ----------
FROM golang:1.22 AS build
WORKDIR /src
COPY go.mod main.go ./
RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /out/inventory .

# ---------- Stage 2: ship only the binary ----------
FROM scratch
COPY --from=build /out/inventory /inventory
EXPOSE 8080
ENTRYPOINT ["/inventory"]
```

PART	WHAT IT MEANS
`FROM golang:1.22 AS build`	Builder with the full Go toolchain (~850 MB). Only used to compile.
`CGO_ENABLED=0`	Build pure Go, with no dependency on the C library (libc). Result: a **static** binary that runs on an empty filesystem. **Required for scratch.**
`-ldflags="-s -w"`	Strip the symbol table (`-s`) and debug info (`-w`). Smaller binary, same behaviour.
`-o /out/inventory .`	Write the compiled program to `/out/inventory` ; `.` = compile the code in this folder.

PART	WHAT IT MEANS
`FROM scratch`	Start from a **completely empty** image: no OS, no shell, no `ls`, nothing.
`COPY --from=build /out/inventory /inventory`	The only thing in the final image: our 5.5 MB program.
`ENTRYPOINT ["/inventory"]`	Run the program when the container starts. `ENTRYPOINT` is the fixed main command; `CMD` is a default that's easy to override.

STEP 5 Build the image

terminal · inside inventory — POWERSHELL / BASH

```
docker build -t shopease-inventory:1.0 .
```

EXPECTED OUTPUT (TRIMMED)

```
[+] Building 21.7s (10/10) FINISHED
 => [build 1/4] FROM docker.io/library/golang:1.22
 => [build 2/4] WORKDIR /src
 => [build 3/4] COPY go.mod main.go ./
 => [build 4/4] RUN CGO_ENABLED=0 go build -ldflags="-s -w" -o /out/inventory .
 => [stage-1 1/1] COPY --from=build /out/inventory /inventory
 => exporting to image
 => => naming to docker.io/library/shopease-inventory:1.0
```

STEP 6 Compare the sizes

terminal — POWERSHELL / BASH

```
docker images golang
docker images shopease-inventory
```

EXPECTED OUTPUT — sizes approximate

```
REPOSITORY          TAG   IMAGE ID       CREATED          SIZE
golang              1.22  3d4e5f6a7b8c   8 months ago     ~850MB

REPOSITORY          TAG   IMAGE ID       CREATED          SIZE
shopease-inventory  1.0   8f9e0d1c2b3a   15 seconds ago   5.53MB
```

CHECKPOINT

Your inventory image should be **under 15 MB** (typically ~5–6 MB). That's more than **99% smaller** than the image used to build it.

STEP 7 Run it and test in the browser

terminal — POWERSHELL / BASH

```
docker run -d --name inv -p 8080:8080 shopease-inventory:1.0
docker logs inv
```

EXPECTED OUTPUT

```
3a7c9e...e41b                    <- the new container's ID
2026/10/17 11:15:49 inventory listening on :8080
```

Open `http://localhost:8080/stock` in your browser (or `curl.exe http://localhost:8080/stock`):

EXPECTED IN BROWSER

```
{"laptop-stand":42,"usb-c-hub":17}
```

STEP 8 See why scratch is secure: there's nothing to break into

terminal — POWERSHELL / BASH

```
docker exec -it inv sh
```

EXPECTED OUTPUT · THIS ERROR IS THE POINT

```
OCI runtime exec failed: exec failed: unable to start container process:
exec: "sh": executable file not found in $PATH
```

CONCEPT · NO SHELL = LESS TO ATTACK (AND LESS TO DEBUG)

There's no shell, no package manager and no tools in a scratch image. Even if someone found a bug in the program, they'd have nothing to run. The trade-off: you debug with `docker logs` instead of "exec-ing in". Two more things scratch doesn't have: CA certificates (so outgoing HTTPS calls need certs copied in) and timezone data.

IF LAB 2 FAILS

`exec /inventory: no such file or directory` (container exits immediately, seen in `docker logs inv`)	The binary was built without `CGO_ENABLED=0`, so it needs libc, which scratch doesn't have. Fix the `RUN` line and rebuild.
`go: go.mod requires go >= 1.23 (running go 1.22...)`	The version in `go.mod` is newer than the builder image. Set it to `go 1.22`.
`syntax error: unexpected newline` / `missing function body`	A `{` was moved onto its own line in `main.go`. Put it back at the end of the `func(...)` line.
`COPY failed: ... "/main.go": not found`	Terminal is in the wrong folder, or file is named `Main.go` / `main.go.txt`.
`Bind for 0.0.0.0:8080 failed: port is already allocated`	Something else uses 8080. Use another host port: `-p 8081:8080`, then browse to `localhost:8081/stock`.
`Conflict. The container name "/inv" is already in use`	An old container with that name exists. Remove it: `docker rm -f inv`.

PART 06 · SESSION Q&A

"How do I turn my running web app into an image?"

A learner (Ankush) asked: if I want to run my web app in a Docker container and create an image from it, how do I do it? There are two ways. The session covered the snapshot way; production uses the Dockerfile way.

Way 1 · Snapshot a running container with docker commit

1. Download a base image (OS + runtime).
2. Create a container from it.
3. Inside the container, deploy / run / test your app and install the third-party tools it needs (like git).
4. Commit the container with `docker commit`. This creates an image from the running container, like a snapshot.
5. Push the image to Docker Hub.

terminal — POWERSHELL / BASH

```
# 1-2. Start a container from a base image and get a shell inside it
docker run -it --name mybox ubuntu:24.04 bash

# 3. (now INSIDE the container) install what you need, set up your app
apt-get update && apt-get install -y git
exit

# 4. Snapshot the container into an image
docker commit mybox <your-dockerhub-username>/mybox:1.0

# 5. Log in and push to Docker Hub
docker login
docker push <your-dockerhub-username>/mybox:1.0
```

EXPECTED OUTPUT (COMMIT AND PUSH)

```
sha256:4f1e2d3c…
The push refers to repository [docker.io/<your-dockerhub-username>/mybox]
5d6e7f8a9b0c: Pushed
1.0: digest: sha256:… size: 741
```

Way 2 · Write a Dockerfile (what we did all session)

terminal — POWERSHELL / BASH

```
docker build -t <your-dockerhub-username>/shopease-api:1.0 -f Dockerfile.multi .
docker login
docker push <your-dockerhub-username>/shopease-api:1.0
```

Already built an image with a local name? Give it a Docker Hub name with `docker tag shopease-api:multi <user>/shopease-api:1.0`, then push.

	DOCKER COMMIT	DOCKERFILE
Repeatable?	No Nobody knows exactly what was typed inside	Yes Same file, same image, every time
Reviewable in Git?	No It's a binary snapshot	Yes Plain text, code-reviewed like any code
Size	Usually large: apt caches, temp files, bash history all included	Small with slim bases and multi-stage builds
Security	Can silently capture secrets you typed or files you left behind	Controlled: only what you `COPY`
Good for	Quick experiments, saving a debugging state	Everything that goes to CI/CD and production

IF IT FAILS

- `denied: requested access to the resource is denied`: you're not logged in, or the image name doesn't start with *your* Docker Hub username. Re-tag it and run `docker login`.
- `An image does not exist locally with the tag`: the name you push doesn't exactly match the name you built or tagged. Check with `docker images`.

PART 07 · IMAGE HARDENING

Building secure images: the Ten Rules

Lean was about size. Hardening is about limiting damage: if the app gets hacked, the attacker should land in a box with no tools, no power and no way to make things worse.

#	RULE	WHAT IT MEANS	HOW
1	Minimal base image	Slim, alpine or distroless. Less software means fewer vulnerabilities.	`FROM python:3.12-slim`
2	Pin versions	Use `python:3.12-slim`, never `latest`. Experts also pin the digest for exact repeatability.	`FROM python:3.12-slim@sha256:…`
3	Run as non-root	Root inside a container is dangerous if the app is compromised.	`USER app`
4	Install only what you need	No curl, vim or git in production images.	Multi-stage; slim bases
5	Read-only filesystem	Stops attackers writing tools or changing files at runtime.	`--read-only`
6	Drop Linux capabilities	Remove all "root superpowers", add back only what's required.	`--cap-drop ALL`
7	No privilege escalation	Programs can't gain more privileges than they started with.	`--security-opt no-new-privileges:true`
8	Limit resources	Stop one container starving the whole host.	`--memory --cpus --pids-limit`
9	Scan images	Trivy or Docker Scout, in CI, on every build.	`trivy image …`
10	Add a HEALTHCHECK	Orchestrators know when the app is truly healthy, not just "running".	`HEALTHCHECK CMD …`

BUILD TIME · DOCKERFILE

Rules 1, 2, 3, 4, 10

slim base · pinned tag

non-root USER · HEALTHCHECK

Lab 3

RUN TIME · DOCKER RUN

Rules 5, 6, 7, 8

read-only · cap-drop ALL

no-new-privileges · limits

Lab 4

PIPELINE · CI/CD

Rule 9

scan every build

fail on HIGH / CRITICAL

Lab 5

Hardening happens in three places. Each layer still protects you if another one is missed: this is called defence in depth.

Bonus rule from the "production view": never put secrets in images

NEVER DO THIS

Dockerfile (BAD example) — DOCKERFILE

```
ENV AWS_SECRET_ACCESS_KEY=abc123...      # visible to anyone with the image
COPY .env .                              # secret file baked into a layer
```

Anyone who can pull the image can read these with `docker history` or by extracting the layers, even if you delete the file in a later step (remember: layers never forget).

DO THIS INSTEAD

- Add `.env` and key files to `.dockerignore` so they never enter the build context.
- Pass secrets at **run** time: `docker run --env-file .env …`, or from a secret manager (AWS Secrets Manager, Kubernetes Secrets).
- If a build step truly needs a secret (e.g. a private package token), use BuildKit secrets: `RUN --mount=type=secret,id=token …`. The secret is available during that one step and is never saved in a layer.

PART 08 · LAB 3

Build the hardened Product API image

Take the lean multi-stage image and add a non-root user, clean Python settings and a health check.

STEP 1 Go back to the product-api folder

terminal — POWERSHELL / BASH

```
cd ..\product-api    # Windows PowerShell
cd ../product-api    # macOS / Linux
```

STEP 2 Create Dockerfile.hardened

product-api/Dockerfile.hardened — DOCKERFILE

```
# ---------- Stage 1: builder ----------
FROM python:3.12-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt

# ---------- Stage 2: hardened runtime ----------
FROM python:3.12-slim
ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1
RUN groupadd --system app && useradd --system --gid app --no-create-home app
WORKDIR /app
COPY --from=builder /install /usr/local
COPY --chown=app:app app.py .
USER app
EXPOSE 8000
HEALTHCHECK --interval=15s --timeout=3s --retries=3 \
  CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8000/health')" || exit 1
CMD ["gunicorn", "-b", "0.0.0.0:8000", "app:app"]
```

COPY-PASTE WARNING · LINES THAT MUST NOT BE SPLIT

In the session notes, two lines wrapped onto a second line. Docker reads every new line as a new instruction, so a wrapped line breaks the build. Make sure these are each on **one line** (or end with `\` to continue):

- `RUN groupadd ... --no-create-home app`: the final word `app` must be on the same line. If it lands alone on the next line you get `unknown instruction: app`.
- The `CMD python -c "import urllib.request; urllib.request.urlopen(...)" || exit 1` line under `HEALTHCHECK` must be one line. Otherwise: `unknown instruction: URLLIB.REQUEST.URLOPEN(...)`.

Line by line: what's new compared with Dockerfile.multi

LINE	WHAT IT DOES AND WHY
`ENV PYTHONDONTWRITEBYTECODE=1`	Prevents Python from writing `.pyc` bytecode files when importing modules. Nothing needs to be written to disk, which suits the read-only filesystem in Lab 4.

LINE	WHAT IT DOES AND WHY
`ENV PYTHONUNBUFFERED=1`	Disables buffering of Python's stdout and stderr, so logs appear in `docker logs` immediately instead of in delayed batches.
`RUN groupadd --system app && useradd --system --gid app --no-create-home app`	Creates a group `app` and a **system** user `app` in it: no password, no login shell, no home folder. A user that exists only to run the app.
`COPY --chown=app:app app.py .`	Copies the code and makes `app` its owner.
`USER app`	**Everything after this line, including the running app, runs as `app`, not root.**
`HEALTHCHECK --interval=15s --timeout=3s --retries=3`	Every 15 s Docker runs the check; if it takes longer than 3 s it counts as a failure; after 3 failures in a row the container is marked `unhealthy`.
`CMD python -c "import urllib.request; ..." || exit 1`	The check itself: call `/health` using Python's built-in library (no need to install curl, which follows rule 4). If the call fails, exit with code 1 = unhealthy.

STEP 3 Build the hardened image

terminal · inside product-api — POWERSHELL / BASH

```
docker build -f Dockerfile.hardened -t shopease-api:hardened .
docker images shopease-api
```

EXPECTED OUTPUT — sizes approximate

```
REPOSITORY     TAG        IMAGE ID       CREATED          SIZE
shopease-api   hardened   a9b8c7d6e5f4   5 seconds ago    ~140MB
shopease-api   multi      7e1d2c3b4a59   20 minutes ago   ~140MB
shopease-api   single     2b3c4d5e6f70   21 minutes ago   ~1.04GB
```

Hardening costs almost nothing in size. Security doesn't have to mean bigger.

STEP 4 Run the naive and hardened images side by side

terminal — POWERSHELL / BASH

```
docker run -d --name api-single   -p 8002:8000 shopease-api:single
docker run -d --name api-hardened -p 8000:8000 shopease-api:hardened
```

Test both: `http://localhost:8002/products` (single) and `http://localhost:8000/products` (hardened). Both should return the same product JSON.

STEP 5 Who is the app running as?

terminal — POWERSHELL / BASH

```
docker exec api-single   whoami
docker exec api-hardened whoami
docker exec api-hardened id
```

EXPECTED OUTPUT — uid/gid number may differ

```
root
app
uid=999(app) gid=999(app) groups=999(app)
```

CHECKPOINT

Single image runs as `root`, hardened image runs as `app`. A hacked app in the hardened container can't install packages, can't modify system files, and can't read files owned by root.

STEP 6 Watch the health check

terminal — POWERSHELL / BASH

```
docker ps
docker inspect --format "{{.State.Health.Status}}" api-hardened
```

EXPECTED OUTPUT (AFTER ~15–20 SECONDS)

```
IMAGE                  STATUS                   PORTS                    NAMES
shopease-api:hardened  Up 25 seconds (healthy)  0.0.0.0:8000->8000/tcp  api-hardened
shopease-api:single    Up 26 seconds            0.0.0.0:8002->8000/tcp  api-single

healthy
```

The single container has no health status at all: Docker only knows it's "Up", not whether it actually works. In the first ~15 seconds you'll see `(health: starting)`; that's normal.

IF LAB 3 FAILS

`unknown instruction: app` / `unknown instruction: URLLIB.REQUEST.URLOPEN(...)`	A line got split while copying (see the warning above). Join it back into one line.
`Bind for 0.0.0.0:8000 failed: port is already allocated`	Something already uses 8000 (maybe an old container). `docker ps` → `docker rm -f <name>`.
Status stays `(unhealthy)`	See what the check says: `docker inspect --format "{{json .State.Health}}" api-hardened`. Usually the port in the HEALTHCHECK doesn't match the Gunicorn port, or the app crashed (`docker logs api-hardened`).
`whoami` prints `root` for the hardened container	`USER app` is missing, or you ran the wrong image. Check with `docker inspect --format "{{.Config.User}}" shopease-api:hardened` (should print `app`).
`useradd: group 'app' does not exist`	The `groupadd` part is missing or misspelled. Both commands must be in the same `RUN` , joined by `&&` .

PART 09 · LAB 4

Lock the container down at runtime

The image is hardened. Now restrict what the running container is allowed to do, using `docker run` flags (rules 5–8).

STEP 1 Remove the hardened container to free port 8000

terminal — POWERSHELL / BASH

```
docker rm -f api-hardened
```

Keep `api-single` running: we'll use it for comparison.

STEP 2 Start the locked-down container

Pick the version for your terminal. The command is long, so it's split across lines, and **each shell uses a different line-continuation character**.

Windows PowerShell · backtick ` at line end — POWERSHELL

```
docker run -d --name api-locked -p 8000:8000 `
  --read-only --tmpfs /tmp `
  --cap-drop ALL `
  --security-opt no-new-privileges:true `
  --memory 256m --cpus 0.5 --pids-limit 100 `
  shopease-api:hardened
```

macOS / Linux / WSL / Git Bash · backslash \ at line end — BASH

```
docker run -d --name api-locked -p 8000:8000 \
  --read-only --tmpfs /tmp \
  --cap-drop ALL \
  --security-opt no-new-privileges:true \
  --memory 256m --cpus 0.5 --pids-limit 100 \
  shopease-api:hardened
```

WINDOWS · THE #1 ERROR IN THIS LAB

Pasting the `\` version into PowerShell fails, because PowerShell doesn't treat `\` as "continue on next line". You'll see `docker: invalid reference format` followed by `--read-only : The term '--read-only' is not recognized…`. Use the backtick version, or put the whole command on one line (that's why the session also showed a one-line version). Also: the backtick must be the **very last character** on the line, with no space after it.

What each flag does

FLAG	MEANING
`--read-only`	The container's whole filesystem becomes read-only. An attacker can't drop a script, change your code or install tools. **(Rule 5)**

FLAG	MEANING
`--tmpfs /tmp`	Gives the app one small writable folder, `/tmp`, in memory only (wiped when the container stops). Gunicorn needs it for its worker heartbeat files.
`--cap-drop ALL`	Removes every Linux capability (see below). Our app on port 8000 needs none. **(Rule 6)**
`--security-opt no-new-privileges:true`	Processes can never gain more privileges than they started with, even via setuid programs like `su` or `passwd`. **(Rule 7)**
`--memory 256m`	Max 256 MB of RAM. Go over it and the container is killed (OOM), not your server. **(Rule 8)**
`--cpus 0.5`	Max half of one CPU core.
`--pids-limit 100`	Max 100 processes. Stops "fork bomb" attacks that spawn processes until the host freezes.

CONCEPT · LINUX CAPABILITIES IN ONE PARAGRAPH

Linux splits root's power into about 40 separate "superpowers" called **capabilities**: e.g. `NET_BIND_SERVICE` (use ports below 1024), `CHOWN` (change file owners), `NET_RAW` (craft raw network packets), `SYS_ADMIN` (almost everything). Docker gives containers about 14 of them by default. `--cap-drop ALL` removes them all. If your app really needs one, add just that one back, e.g. `--cap-add NET_BIND_SERVICE` to listen on port 80.

STEP 3 Confirm it's running and healthy

terminal — POWERSHELL / BASH

```
docker ps --filter name=api-locked
curl.exe http://localhost:8000/health
```

EXPECTED OUTPUT

```
IMAGE                  STATUS                    PORTS                    NAMES
shopease-api:hardened  Up 30 seconds (healthy)   0.0.0.0:8000->8000/tcp   api-locked

{"status":"ok"}
```

STEP 4 Test the read-only filesystem (pretend to be an attacker)

terminal — POWERSHELL / BASH

```
docker exec api-locked touch /app/hacked.txt
docker exec api-locked touch /tmp/scratch.txt
docker exec api-single touch /app/hacked.txt
```

EXPECTED OUTPUT

```
touch: cannot touch '/app/hacked.txt': Read-only file system
(no output: /tmp is the one writable, in-memory folder)
(no output: the naive container happily let us write a file)
```

CHECKPOINT

The locked container refuses writes everywhere except `/tmp`. The single container, running as root with a writable filesystem, accepts anything, which is exactly what an attacker wants.

STEP 5 Verify the limits Docker applied

terminal — POWERSHELL / BASH

```
docker inspect api-locked --format "ReadOnly={{.HostConfig.ReadonlyRootfs}}"
docker inspect api-locked --format "CapDrop={{.HostConfig.CapDrop}}"
docker inspect api-locked --format "SecOpt={{.HostConfig.SecurityOpt}}"
docker inspect api-locked --format "Memory={{.HostConfig.Memory}} Pids={{.HostConfig.PidsLimit}}"
docker inspect api-locked --format "NanoCpus={{.HostConfig.NanoCpus}}"
docker stats --no-stream api-locked
```

EXPECTED OUTPUT — usage numbers will differ

```
ReadOnly=true
CapDrop=[ALL]
SecOpt=[no-new-privileges:true]
Memory=268435456 Pids=100
NanoCpus=500000000

CONTAINER ID   NAME         CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O   PIDS
e4f5a6b7c8d9   api-locked   0.05%     38.2MiB / 256MiB    14.92%    2.1kB / 1kB   0B / 0B     2
```

268435456 bytes = 256 MB. 500000000 nano-CPUs = 0.5 CPU.

IF LAB 4 FAILS

`docker: invalid reference format` + `The term '--read-only' is not recognized`	Bash-style `\` used in PowerShell. Use the backtick version or one line.
Same errors even with backticks	There's a space after a backtick. Delete trailing spaces, or use one line.
`Conflict. The container name "/api-locked" is already in use`	A previous attempt half-worked. `docker rm -f api-locked` and run again.
`port is already allocated`	You didn't remove `api-hardened` (Step 1).
Container exits; logs show `No usable temporary directory found in ['/tmp', '/var/tmp', '/usr/tmp', '/app']`	You forgot `--tmpfs /tmp`. Gunicorn needs one writable temp folder.
Container keeps dying, `docker inspect --format "{{.State.OOMKilled}}" api-locked` prints `true`, exit code 137	Memory limit too low for the app. Raise `--memory`.
`touch` says `Permission denied` instead of `Read-only file system`	You ran it against a container without `--read-only` (e.g. `api-hardened`). It's still blocked, because the non-root user doesn't own `/app`. Two protections, two different messages.

PART 10 · LAB 5

Scan for vulnerabilities with Trivy

Every image contains hundreds of packages, and some will have known security bugs (CVEs). Trivy compares what's in your image against a public vulnerability database and tells you what to fix.

STEP 1 Run the scan

We run Trivy itself as a container, so there's nothing to install.

Windows PowerShell — POWERSHELL

```
docker run --rm `
  -v /var/run/docker.sock:/var/run/docker.sock `
  aquasec/trivy:latest image --severity HIGH,CRITICAL shopease-api:hardened
```

macOS / Linux / WSL — BASH

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  aquasec/trivy:latest image --severity HIGH,CRITICAL shopease-api:hardened
```

PART	MEANING
`docker run --rm`	Run Trivy in a throwaway container, deleted when the scan ends.
`-v /var/run/docker.sock:/var/run/docker.sock`	Lets Trivy (inside its container) talk to your Docker engine, so it can find and read the `shopease-api:hardened` image that lives on your machine.
`aquasec/trivy:latest`	The Trivy image we pre-pulled.
`image`	Trivy mode: scan a container image.
`--severity HIGH,CRITICAL`	Only show the serious ones (skip LOW and MEDIUM) so the report is actionable.
`shopease-api:hardened`	The image to scan.

STEP 2 Read the report

EXPECTED OUTPUT (SHAPE) — package names and counts change daily

```
INFO  [vulndb] Need to update DB
INFO  [vulndb] Downloading vulnerability DB...
INFO  Detected OS  family="debian" version="13.x"
INFO  [debian] Detecting vulnerabilities...
INFO  [python-pkg] Detecting vulnerabilities...

Report Summary
+---------------------------------------+------------+-----------------+
| Target                                | Type       | Vulnerabilities |
+---------------------------------------+------------+-----------------+
| shopease-api:hardened (debian 13.x)   | debian     |        N        |
| usr/local/lib/python3.12/site-packages | python-pkg |        N        |
+---------------------------------------+------------+-----------------+

shopease-api:hardened (debian 13.x)
Total: N (HIGH: n, CRITICAL: n)

+------------+----------------+----------+----------+-----------+---------+
| Library    | Vulnerability  | Severity | Status   | Installed | Fixed   |
+------------+----------------+----------+----------+-----------+---------+
| libexample | CVE-YYYY-NNNNN | HIGH     | fixed    | 1.2.3-1   | 1.2.3-2 |
| libother   | CVE-YYYY-NNNNN | CRITICAL | affected | 4.5.6-1   |         |
+------------+----------------+----------+----------+-----------+---------+
(Trivy draws these with box lines; columns trimmed here, the real table also has a Title column)
```

"Total: 0" is a great result, but don't panic if you see a few: brand-new CVEs are published every day, so the same image can scan clean today and show findings next week.

COLUMN	HOW TO READ IT
Target / Type	`debian` = OS packages from the base image. `python-pkg` = libraries you installed with pip.
Library	The package that has the problem.
Vulnerability	The CVE ID. Search it to read the details.
Severity	CRITICAL > HIGH > MEDIUM > LOW. Fix critical and high first.
Status	`fixed` = a patched version exists, so you can fix it now. `affected` / `will_not_fix` = no fix yet; track it and accept or mitigate the risk.
Installed / Fixed	Your version vs the first version that fixes it.

STEP 3 Compare with the naive image

terminal (PowerShell shown; use \ on bash) — POWERSHELL

```
docker run --rm `
  -v /var/run/docker.sock:/var/run/docker.sock `
  aquasec/trivy:latest image --severity HIGH,CRITICAL shopease-api:single
```

CHECKPOINT

The single image (full Debian with compilers, git, curl and hundreds more packages) will report **many times more** HIGH/CRITICAL findings than the hardened image. Same app, far bigger attack surface. That's rules 1 and 4 in action.

STEP 4 Fix what Trivy finds

1. **Python packages** (`python-pkg`): bump the version in `requirements.txt` to the "Fixed" version or newer, then rebuild.
2. **OS packages** (`debian`): rebuild with a fresh base image, since official images are patched regularly: `docker build --pull --no-cache -f Dockerfile.hardened -t shopease-api:hardened .`
3. **No fix available**: consider a smaller base (fewer packages = fewer findings), or document and accept the risk. Trivy can skip these with `--ignore-unfixed` .
4. Scan again to confirm.

GOING FURTHER · SCANNING IN CI ON EVERY BUILD (RULE 9)

Add `--exit-code 1` and Trivy returns an error code when it finds HIGH/CRITICAL issues. In Jenkins, GitHub Actions or GitLab CI that fails the pipeline, so a vulnerable image never reaches production. Docker Desktop users can also try `docker scout quickview shopease-api:hardened` for a similar report.

IF LAB 5 FAILS

`unable to find the specified image "shopease-api:hardened"` / `Cannot connect to the Docker daemon`	The `-v /var/run/docker.sock:...` part is missing or mistyped, so Trivy can't see your local images. Check the image name with `docker images` .
Same error in Git Bash on Windows	Git Bash rewrites paths. Use a double slash: `-v //var/run/docker.sock:/var/run/docker.sock` , or use PowerShell.
First scan is slow / sits on "Downloading vulnerability DB"	Normal: it downloads the database the first time. To keep it between scans, add a cache volume: `-v trivy-cache:/root/.cache/` .
`TOOMANYREQUESTS` while downloading the DB	The DB server is rate-limiting. Wait a few minutes and retry.
PowerShell errors on the multi-line command	Same backtick rules as Lab 4, or put it on one line.

PART 11

Troubleshooting index

Every error you're likely to hit in this lab, in one place. Find the message you see in the left column.

First, three commands that diagnose 90% of problems

COMMAND	USE IT WHEN
`docker ps -a`	Your container "disappeared". `-a` shows stopped containers and their exit codes too.
`docker logs <name>`	Anything crashed or misbehaves. The app's error message is almost always here.
`docker inspect <name>`	You want to see the exact config: user, ports, limits, health, mounts.

Docker setup

YOU SEE	CAUSE → FIX
`error during connect … dockerDesktopLinuxEngine` / `Cannot connect to the Docker daemon`	Docker Desktop isn't running → start it, wait for "Engine running".
`'docker' is not recognized`	Not installed or terminal opened before install → install / restart VS Code.
`toomanyrequests: … pull rate limit`	Anonymous Docker Hub limit → `docker login`, retry.
`no space left on device`	Disk full → `docker system df` to check, `docker system prune` to clean.

Building images

YOU SEE	CAUSE → FIX
`failed to read dockerfile: open …: no such file or directory`	Wrong folder or wrong file name (e.g. `.txt` added) → `cd` to the right folder, check the name.
`"docker build" requires exactly 1 argument`	Missing the final `.` → add it.
`COPY failed: … not found`	File missing, wrong case (`App.py`), or excluded by `.dockerignore`.
`unknown instruction: …`	A line got split when copying → join it, or end the first part with `\`.
`Could not find a version that satisfies the requirement`	Typo in `requirements.txt` (use `==`) or no network.
`go: go.mod requires go >= …`	`go.mod` version newer than builder → set `go 1.22`.
`useradd: group 'app' does not exist`	`groupadd` missing in the same `RUN`.

Running containers

YOU SEE	CAUSE → FIX
`port is already allocated`	Port in use → `docker ps`, `docker rm -f <name>`, or choose another host port.
`The container name "/x" is already in use`	Old container → `docker rm -f x`.
`Failed to find attribute 'app' in 'app'` / `No module named 'app'`	File or variable not named `app`.
`exec /inventory: no such file or directory`	Go binary not static → `CGO_ENABLED=0`.
`exec: "sh": executable file not found in $PATH`	Expected on scratch; there is no shell. Use `docker logs`.
`ERR_CONNECTION_REFUSED` in browser	Wrong port, missing `-p`, container stopped, or app bound to `127.0.0.1` instead of `0.0.0.0`.
PowerShell `curl` prints a big object	It's `Invoke-WebRequest` → use `curl.exe`.

Hardening and lockdown

YOU SEE	CAUSE → FIX
`(health: starting)` for a while	Normal for the first interval (15 s). Wait.
`(unhealthy)`	`docker inspect --format "{{json .State.Health}}" <name>` → usually port mismatch or app crash.
`invalid reference format` + `'--read-only' is not recognized`	Bash `\` used in PowerShell → use backtick `` ` `` or one line; no space after the backtick.
`No usable temporary directory found`	Missing `--tmpfs /tmp` with `--read-only`.
Exit code 137, `OOMKilled=true`	Memory limit too low → increase `--memory`.
`Permission denied` vs `Read-only file system`	Both mean "blocked". The first comes from the non-root user, the second from `--read-only`.

Trivy

YOU SEE	CAUSE → FIX
`unable to find the specified image`	Socket not mounted, or image name typo → check `-v` and `docker images`.
Git Bash: same error with correct command	Path rewriting → `-v //var/run/docker.sock:/var/run/docker.sock`.
Very slow first run	DB download → add `-v trivy-cache:/root/.cache/` to reuse it.
`TOOMANYREQUESTS`	DB server rate limit → wait and retry.

PART 12

Cheat sheet, cleanup and practice

Everything from the session on one page, plus exercises to lock in what you learned.

Commands at a glance

COMMAND	WHAT IT DOES
`docker pull <image:tag>`	Download an image
`docker images <repo>`	List images and their sizes
`docker build -f <file> -t <name:tag> .`	Build an image from a specific Dockerfile
`docker run -d --name <n> -p HOST:CTR <image>`	Start a container in the background with a port mapping
`docker ps` / `docker ps -a`	Running containers / all containers
`docker logs <name>`	See the container's output
`docker exec <name> <cmd>`	Run a command inside a running container
`docker inspect --format "{{…}}" <name>`	Read one setting (user, health, limits...)
`docker stats --no-stream`	CPU / memory / PIDs usage snapshot
`docker rm -f <name>`	Stop and delete a container
`docker rmi <image>`	Delete an image
`docker commit <ctr> <image>`	Snapshot a container into an image
`docker tag <src> <user>/<repo>:<tag>`	Give an image a registry name
`docker login` / `docker push`	Authenticate / upload to Docker Hub
`docker system df` / `docker system prune`	Check / reclaim Docker disk space

Cleanup

When you're done, free up ports and disk space:

terminal — POWERSHELL / BASH

```
docker rm -f api-single api-locked api-hardened api-multi inv
docker rmi shopease-api:single shopease-api:multi shopease-api:hardened shopease-inventory:1.0
docker image prune -f
```

"No such container" messages for containers you already removed are harmless. Keep the base images (python, golang, trivy) if you plan to practice; they're the slow downloads.

Hardening checklist for your own images

- [ ] Base image is slim / alpine / distroless / scratch
- [ ] Base tag is pinned (no `latest`)
- [ ] Multi-stage build; no compilers in the final image
- [ ] No curl / vim / git in the final image
- [ ] `USER` is a non-root user
- [ ] No secrets in `ENV`, `ARG` or copied files; `.dockerignore` exists
- [ ] `HEALTHCHECK` defined
- [ ] Runs with `--read-only`, `--cap-drop ALL`, `no-new-privileges`
- [ ] Memory, CPU and PID limits set
- [ ] Trivy scan passes in CI (no HIGH/CRITICAL with a fix available)

Practice tasks

#	TASK	HINT
1	Make the Go inventory image non-root too. Prove it without `whoami` (there's no shell!).	Add `USER 65534:65534` after the `COPY`; check with `docker inspect --format "{{.Config.User}}"`.
2	Build the Product API on `python:3.12-alpine` and compare its size and Trivy results with slim.	Change both `FROM` lines; on Alpine use `addgroup -S` / `adduser -S`.
3	Make Trivy fail like a CI pipeline would.	Add `--exit-code 1`, then check `$LASTEXITCODE` (PowerShell) or `$?` (bash).
4	Start the locked container with `--memory 20m`. What happens and why?	`docker ps -a`, then inspect `.State.OOMKilled`.
5	Pin the slim base image by digest.	`docker inspect --format "{{index .RepoDigests 0}}" python:3.12-slim`, then use it in `FROM`.
6	Push your hardened image to Docker Hub and pull it on another machine.	`docker tag` → `docker login` → `docker push`.

Session recap

WHAT	BEFORE	AFTER
Python API image	~1.04 GB full base, single stage	~140 MB slim + multi-stage
Go service image	~850 MB toolchain image	5.5 MB static binary on scratch
Runs as	root	app (non-root system user)
Filesystem	writable	read-only + in-memory /tmp
Privileges	~14 capabilities	none + no-new-privileges
Resources	unlimited	256 MB · 0.5 CPU · 100 PIDs
Health	"Up" only	healthy via HEALTHCHECK
Vulnerabilities	unknown	scanned with Trivy